Verified Projection-Event Calculus in Lean 4: Bundled Arbitrary-Partition Pinching, Lüders Retractions, and CHSH Interoperability

Bernhard Mueller*
Pragma Research Inc., Washington, United States

July 21, 2026

Paper release: `r1570` **Released:** July 21, 2026

Abstract

Finite quantum measurement looks simple on paper: multiply matrices, take a trace, and normalize. In Lean, each step crosses a different interface and every side condition must be supplied. We build a Lean 4 library for projection events over finite complex matrix algebras. A projective partition defines a star-subalgebra of matrices that commute with its projectors and a complex-linear pinching map onto that subalgebra. The map is unital, positive, trace preserving, and idempotent. Its range and fixed points are exactly the partition commutant. The library also proves the bimodule law, Hilbert–Schmidt Pythagorean and contraction identities, and a trace-duality uniqueness theorem. Lüders conditioning is available as a total matrix formula and as a typed state update that requires a nonzero outcome weight. The fixed states are exactly the states assigning weight one to the event. Pinching commutes with the typed update when the event belongs to the commutant. A final adapter turns four projection events into dichotomic observables and applies an operator-norm Tsirelson theorem through Mathlib's Clauser-Horne-Shimony-Holt (CHSH) interface. We compare it with Mathlib, Physlib, Lean-Quantum, Lean-QIT, and Isabelle/HOL. The artifact pins its Lean and Mathlib revisions and records declaration-level axiom dependencies. Complete positivity, a Physlib correspondence, rank-one classicality, and a state-expectation CHSH bound are outside the formalization.

Keywords: formal verification, Lean 4, projective measurement, partition pinching, Lüders update, CHSH

Mathematics Subject Classification (2020): Primary 68V20; Secondary 81P15, 81P40, 46L53.

1 Introduction

Finite quantum measurement is short on paper. A proof calls a matrix a state, inserts a projection, normalizes the result, and moves on. Lean asks for every condition skipped in that sentence. The

*Corresponding author: bernhard@floatingpragma.ai

matrix must be positive and have unit trace. The projection must be Hermitian and idempotent. Normalization needs a nonzero denominator. Later norm estimates require yet another set of instances.

The library packages these conditions into a finite projection-event API. Its main input is an arbitrary projective partition of the identity. The associated pinching deletes off-block terms and maps the matrix algebra onto the partition commutant. The same event definitions feed the Lüders update and the CHSH adapter. This keeps algebraic identities separate from results that use trace, positivity, or state normalization.

Lean declaration identifiers are printed throughout in monospaced type, for example `partitionPinching_range`; the underscore is part of the identifier. To locate a declaration, search for the exact printed identifier, after removing only TeX's protective backslash before each underscore, in the `EventAlgebra/*.lean` files. In the public repository these files are under `Lean/ObserverPatchHolography/Source/EventAlgebra/`; in the standalone artifact they are under `event-algebra-lean/EventAlgebra/`. Table 1 maps each module name to its subject matter, and the top-level `EventAlgebra.lean` file imports all five modules.

The library builds on existing formal work. Mathlib contains an order-form CHSH/Tsirelson theorem and the `IsCHSHTuple` interface [4]. Physlib's `QuantumInfo.Finite.Pinching` constructs the spectral pinching of a state as a completely positive trace-preserving map and proves commutation, fixed-point, Pythagorean, and pinching-bound results [6]. Lean-Quantum provides basis-independent infrastructure for states, channels, tensor products, Choi and Kraus representations, and trace inequalities [3]; Lean-QIT develops a compositional infrastructure for quantum-information theory [8]. Zhao and Yu formalize CHSH rigidity in Lean 4 [7]. Isabelle/HOL developments cover projective measurement and the CHSH upper bound [1, 2]. We make no priority claim for projective measurement, pinching, or Tsirelson's inequality.

The submitted code adds five pieces:

(i) an arbitrary supplied projective partition, independent of a selected state or spectral resolution, together with its commutant bundled as a `StarSubalgebra`;

(ii) partition pinching bundled as a complex `LinearMap`, with exact range and fixed-point theorems, positivity, unitality, trace preservation, the commutant bimodule law, Hilbert–Schmidt geometry, and map-level uniqueness;

(iii) a typed nonzero-weight Lüders state update, an unguarded fixed-point characterization for states, and naturality of that typed update under partition pinching for commutant events;

(iv) an event-to-observable CHSH client that discharges Mathlib-compatible selfadjointness, involutivity, and cross-commutation hypotheses before applying a complementary norm-form bound; and

(v) a reproducible, version-pinned artifact with per-declaration axiom output and no proof placeholders.

Four results are outside the current library. The pinching map is not bundled as a completely positive trace-preserving channel. No theorem identifies an arbitrary partition with Physlib's spectral partition. There is no rank-one classical-representation theorem. The CHSH client proves an operator-norm upper bound; it contains no state-expectation bound or sharpness witness.

Section 2 fixes the mathematical and Lean interfaces. Sections 3 and 4 treat projection events and selective update. Section 5 gives the bundled partition construction and its geometry. Sections 6 and 7 describe the state functional and CHSH client. Section 8 reports the audit methodology, and Section 9 positions the development feature by feature.

2 Mathematical and formal setting

Let $\mathrm{M}_n(\mathbb{C})$ be the complex $n \times n$ matrices, with conjugate transpose $X \mapsto X^*$, trace Tr, and identity $\mathbf{1}$. A projection event is a Hermitian idempotent $P = P^* = P^2$. A state is a positive-semidefinite matrix ρ with $\mathrm{Tr}(\rho) = 1$, and the trace pairing is $\mu_\rho(M) = \mathrm{Tr}(\rho M)$.

Lean represents these interfaces as predicates over Mathlib matrices: `IsEvent P` is the conjunction $P^* = P$ and $P^2 = P$; `IsState rho` is positive semidefiniteness and unit trace. For functions that return states, the library defines the subtypes `ProjectionEvent n` and `StateMatrix n`. The split is practical. Algebraic lemmas use raw matrices. A function advertised as a state update returns a value whose type carries the state conditions.

Table 1 lists the five source modules. The comparison in this work concerns their compiled interfaces and uses, not the number of declarations in each file.

Module	Principal interface
`Basic`	projection events, states, trace pairing, closure and Born weight bounds
`Lueders`	totalized matrix formula, typed state update, repeatability, composition, and fixed points
`Partition`	projective partitions, bundled commutant, bundled linear
`Pinching`	pinching, exact range, geometry, bimodule law, and naturality
`StateExpectation`	trace pairing bundled as a complex-linear functional, normalization and positivity
`Tsirelson`	CHSH ring identity, norm theorem, Mathlib interoperability, and event-level matrix client

Table 1: Module structure of the checked development.

Three Mathlib scopes affect compilation. `ComplexOrder` equips $\mathbb{C}$ with the partial order in which $0 \leq z$ means that z is real and nonnegative. `MatrixOrder` supplies the Loewner order. `Matrix.Norms.L2Operator` activates the finite-matrix operator norm and C*-ring instances used by the matrix CHSH theorem. Without the right scope, Lean may report an instance problem even though the required theorem is in scope.

Documentation tags classify results as *algebra-only* or *trace-dependent*. These tags do not define an import hierarchy. They record whether a proof uses ring, star, and norm structure alone, or also uses $\mathrm{Tr}(\rho M)$, positivity, or normalization.

3 Projection events and the trace pairing

The event layer proves the expected closure rules. Zero, identity, and $\mathbf{1}-P$ are events. Orthogonality is symmetric. Orthogonal sums and products of commuting events are events. Subevent absorption holds on both sides, and every event is positive semidefinite. The declaration `IsEvent.one_sub_two_smul_involution` proves that $\mathbf{1}-2P$ is a selfadjoint involution. The CHSH adapter in Section 7 uses this result.

The trace pairing is additive, homogeneous, compatible with finite sums, and obeys the sandwich identity

$$\mathrm{Tr}(P\rho P) = \mathrm{Tr}(\rho P)$$

for idempotent P. Hermiticity of ρ and P implies that the pairing is real. Positivity of ρ and the event property imply $0 \le \mu_\rho(P)$ in Mathlib's order on $\mathbb{C}$; for a state the upper bound is $\mu_\rho(P) \le 1$.

The declaration `isEvent_add_and_bornWeight_add_of_orthogonal` returns two facts: the orthogonal sum is an event, and its Born weight is the sum of the two Born weights. Each orthogonality hypothesis is used to prove the first fact.

4 Lüders update as a partial state interface

For raw matrices, the library uses the totalized formula

$$\mathcal{L}_P(\rho) = \mu_\rho(P)^{-1} P\rho P,$$

Lean's field inverse maps zero to zero. The formula therefore covers algebraic identities in the zero-weight case. Its return type is a raw matrix and carries no state guarantee. The typed operation is

$$\texttt{luedersStateUpdate} : \texttt{StateMatrix n} \to \texttt{ProjectionEvent n} \to (\mu_\rho(P) \neq 0) \to \texttt{StateMatrix n}.$$

This operation assumes `NeZero n`. The unique 0×0 matrix cannot have unit trace, so a state-returning function must exclude that dimension.

Proposition 1 (Checked Lüders interface)**.** *For a state ρ, event P, and nonzero outcome weight:*

(i) the raw formula is positive semidefinite and has trace one (`luedersUpdate_isState`);

(ii) the updated state assigns weight one to P (`bornWeight_luedersUpdate_self`);

(iii) update is an idempotent retraction (`luedersUpdate_idem`);

(iv) updates by commuting events compose and commute (`luedersUpdate_luedersUpdate_of_commute` and `luedersUpdate_comm`); and

(v) the fixed-state equivalence

$$\mathcal{L}_P(\rho) = \rho \quad \Longleftrightarrow \quad \mu_\rho(P) = 1$$

holds without a separate nonzero-weight hypothesis (`luedersUpdate_eq_self_iff`).

The fixed-point theorem needs no nonzero-weight guard. A state fixed by a zero-weight totalized update would equal the zero matrix, which has the wrong trace. The raw function covers the zero-weight case; the state theorem does not carry a redundant premise.

5 Bundled arbitrary-partition pinching

A `ProjectivePartition n k` consists of projections $(P_i)_{i<k}$, pairwise orthogonality, and $\sum_i P_i = \mathbf{1}$. It induces

$$\mathcal{E}_\pi(X) = \sum_i P_i X P_i, \qquad N_\pi = \{X : XP_i = P_i X \text{ for every } i\}.$$

The code defines N_π as `ProjectivePartition.commutant`, a `StarSubalgebra` defined using Mathlib's centralizer, and bundles $\mathcal{E}_\pi$ as `partitionPinchingLinearMap`. The membership theorem `mem_commutant_iff` reduces membership to the pointwise commutation equation.

The range is a block-diagonal commutant and need not be commutative. For this reason, we do not call it a "classical algebra". The library has no theorem identifying the algebra generated by the partition with the center of the commutant. It also has no rank-one classical-representation theorem.

Theorem 1 (Bundled retraction and exact range)**.** *For every projective partition π, the map $\mathcal{E}_\pi$ is complex linear, unital, positive, trace preserving, and idempotent. It lands in N_π and fixes N_π pointwise. Consequently*

$$\mathcal{E}_\pi(X) = X \iff X \in N_\pi, \qquad \operatorname{range}(\mathcal{E}_\pi) = N_\pi$$

where the second equality is the equality of the linear-map range and the underlying submodule of the bundled commutant.

The corresponding declarations are `partitionPinching_unital`, `partitionPinching_posSemidef`, `trace_partitionPinching`, `partitionPinching_idem`, `partitionPinching_eq_self_iff_mem_commutant`, and `range_partitionPinchingLinearMap`. State preservation is exposed by `partitionPinching_isState`.

Theorem 2 (Bimodule and Hilbert–Schmidt structure)**.** *If $A, B \in N_\pi$, then*

$$\mathcal{E}_\pi(AXB) = A\mathcal{E}_\pi(X)B.$$

Pinching is also selfadjoint for the trace pairing and satisfies the Pythagorean identity

$$\operatorname{Tr}(X^*X) = \operatorname{Tr}(\mathcal{E}_\pi(X)^*\mathcal{E}_\pi(X)) + \operatorname{Tr}((X - \mathcal{E}_\pi(X))^*(X - \mathcal{E}_\pi(X))),$$

and hence contracts the squared Hilbert–Schmidt quantity. A commutant-valued complex-linear map with the same trace pairings against every commutant element equals `partitionPinchingLinearMap`.

The corresponding declarations are `partitionPinching_bimodule`, `trace_conjTranspose_partitionPinching_mul`, `trace_partitionPinching_pythagoras`, `trace_partitionPinching_contraction`, and `partitionPinchingLinearMap_unique`. Together they make the map a positive, unital, trace-preserving linear projection with a bimodule law. The library does not instantiate an operator-algebra conditional expectation or a channel interface because complete positivity is not proved.

5.1 Lüders naturality

If $P \in N_\pi$, compression commutes with pinching. After normalization, trace preservation against commutant elements gives

$$\mathcal{E}_\pi(\mathcal{L}_P(\rho)) = \mathcal{L}_P(\mathcal{E}_\pi(\rho)).$$

The raw formula is `partitionPinching_luedersUpdate`. The theorem `partitionPinching_luedersStateUpdate` states the same naturality for `StateMatrix n` and `ProjectionEvent n`: the input and output are states, and the nonzero-weight proof is transported by `bornWeight_partitionPinching`. This checks the same operation through the bundled state and event interfaces.

6 The bundled state expectation

For a matrix ρ, `stateExpectationLinearMap rho` bundles $M \mapsto \mathrm{Tr}(\rho M)$ as a complex-linear functional. If ρ is a state it maps identity to one. If ρ and M are positive semidefinite, then the expectation is nonnegative. The proof of `expectation_nonneg` uses a finite spectral decomposition of M, cycles the trace, and reduces the result to a sum of products of nonnegative diagonal entries. Additivity and homogeneity come from the bundled `LinearMap` interface and are not repeated as pointwise lemmas.

7 CHSH interoperability as a client

For selfadjoint involutions a_0, a_1, b_0, b_1 with cross commutation, set

$$S = a_0 b_0 + a_0 b_1 + a_1 b_0 - a_1 b_1.$$

The development first proves, in a bare ring,

$$S^2 = 4\mathbf{1} - (a_0 a_1 - a_1 a_0)(b_0 b_1 - b_1 b_0)$$

(`chsh_mul_self`). In a nontrivial unital C*-ring this yields the operator-norm bound $\|S\| \le 2\sqrt{2}$ (`tsirelson_bound`). The precise Mathlib structure is a `NormedRing`, `StarRing`, `CStarRing`, and `Nontrivial`. The result is stated for a unital C*-ring.

The theorem `tsirelson_bound_of_isCHSHTuple` accepts Mathlib's `IsCHSHTuple`. The matrix theorem activates Mathlib's scoped operator norm. The declaration `matrix_tsirelson_bound_of_events` accepts four projection events plus the four cross-commutation equations, constructs the dichotomic observables $\mathbf{1} - 2P$, and returns the norm bound. This gives the event layer a compiled path into the CHSH theorem.

Mathlib's `tsirelson_inequality` is an order inequality in a star-ordered real algebra [4]. The theorem in this artifact is an operator-norm bound. It takes no state expectation and provides no equality witness.

8 Artifact evaluation and proof audit

The source is a Lake project. Its toolchain pins `leanprover/lean4:v4.29.1`; its manifest pins Mathlib commit `5e932f97dd25535344f80f9dd8da3aab83df0fe6`. Build the library with

```
lake build EventAlgebra.
```

The canonical Lean modules are publicly available in the Observer Patch Holography repository [5] at commit `da9ff3a551e7c635370ec885d972c9527de173b0`. The manuscript source accompanies the submission. It includes a standalone archive with a byte-identical copy of those modules.

Every public result discussed in the paper appears under its Lean name. The modules end with `#print axioms` commands for the principal declarations. Their output lists propositional extensionality, quotient soundness, classical choice, and other standard Lean/Mathlib axioms. No declaration reports `sorryAx`. A source scan finds no `sorry` placeholder in the submitted modules.

The submitted API uses every orthogonality hypothesis in its orthogonal-sum theorem. The sequential-update theorem has no unused event premise. State update is typed and excludes dimension zero. The fixed-point equivalence has no nonzero-weight guard. The commutant and pinching map are bundled, and the CHSH adapter is a client of the event definitions.

The release archive contains the toolchain, Lake manifest, source modules, build command, checksums, and captured axiom output. It has no public release tag or persistent archive identifier.

9 Feature-level comparison

Table 2 compares compiled features. It does not treat a count of elementary lemmas as a research contribution.

Physlib provides more channel structure: its spectral pinching is a `CPTPMap` [6]. This artifact accepts a supplied partition that need not come from the spectrum of a state. The two APIs therefore have different inputs, and no correspondence theorem connects them.

Lean-Quantum and Lean-QIT cover a larger part of quantum information [3, 8]. Here, the scope is limited to connections among projections, commutants, pinching, selective update, and Mathlib's CHSH interface.

10 Limitations and conclusion

The library connects projection events, Born weights, a typed Lüders update, arbitrary projective partitions, block pinching, and an event-level CHSH norm bound. The pinching map has the bundled commutant as its exact range. It obeys the bimodule law and commutes with typed Lüders update for commutant events. Each statement in the paper points to a declaration in the submitted source.

Complete positivity is not established, so the pinching map is not a quantum channel in the library. There is no theorem connecting it to Physlib's spectral pinching and no rank-one classical specialization. The CHSH client stops at the operator norm. It has no state-expectation corollary

Feature	Closest checked infrastructure	This artifact
Projective measurement	Isabelle/HOL projective-measurement developments; state/measurement interfaces in current Lean libraries	finite matrix event predicate, typed event/state subtypes, closure and trace-pairing lemmas; prerequisite result with no priority claim
Pinching	Physlib spectral pinching is a state-selected bundled CPTP map with fixed-point and geometric results	arbitrary supplied projective partition; bundled linear map and commutant; no CPTP or Physlib correspondence theorem
Retraction structure	operator-algebra conditional expectations and channel interfaces	exact range/fixed points, positive/unital/trace-preserving, bimodule, Hilbert–Schmidt geometry, map-level trace-duality uniqueness
Lüders update	Isabelle projective measurement and general channel infrastructure	raw formula plus typed partial state API, unguarded state fixed-point equivalence, typed naturality with arbitrary-partition pinching
CHSH/ Tsirelson	Mathlib order theorem; Isabelle upper bound; Lean CHSH rigidity	complementary norm theorem and event-level client; no state-level bound or tightness claim
Reproducibility	public CI and archived releases are common best practice	pinned Lean/Mathlib, clean artifact build instructions, checksums and static axiom audit; no DOI claim before deposit

Table 2: Direct comparison with the closest formal infrastructure.

or sharpness witness. The entire development is finite-dimensional and says nothing about normal maps on infinite-dimensional von Neumann algebras.

Open work consists of four items: prove complete positivity, bundle the map as a CPTP channel, connect spectral partitions to Physlib, and add a state-level CHSH client. The present artifact provides the arbitrary-partition map, its exact range and bimodule law, and typed Lüders naturality.

Declarations

Funding. No funding was received for this work.

Competing interests. The author declares no competing interests.

Author contributions. Bernhard Mueller is the sole author, designed the study, reviewed the formal development, and takes responsibility for the manuscript and artifact.

Data and code availability. No empirical data were generated. The Lean source, pinned dependency files, build instructions, axiom report, and checksums accompany the submission. No persistent archive identifier is claimed.

References

[1] Mnacho Echenim. Quantum projective measurements and the CHSH inequality. *Archive of Formal Proofs*, 2021. Formal proof development, ISSN 2150-914x.

[2] Mnacho Echenim and Mehdi Mhalla. A formalization of the CHSH inequality and Tsirelson's

upper-bound in Isabelle/HOL. *Journal of Automated Reasoning*, 68:Article 2, 2024. DOI: 10.1007/s10817-023-09689-9.

[3] Kazumi Kasaura, Kei Tsukamoto, Kento Mori, Risa Mizuno, Takahiro Namatame, Yuta Oriike, Masaya Taniguchi, Sho Sonoda, and Hayata Yamasaki. Lean-Quantum: Toward AI-assisted formalization of quantum information, 2026.

[4] Kim Morrison and Mathlib contributors. Mathlib.algebra.star.chsh. Mathlib documentation, 2026. URL https://leanprover-community.github.io/mathlib4_docs/Mathlib/Algebra/Star/CHSH.html. Accessed 20 July 2026.

[5] Observer Patch Holography contributors. Observer Patch Holography: Source repository. https://github.com/FloatingPragma/observer-patch-holography, 2026. Commit da9ff3a551e7c635370ec885d972c9527de173b0; accessed 20 July 2026.

[6] Physlib contributors. Quantuminfo.finite.pinching: Pinching channels. Lean library documentation, 2026. URL https://ohaithe.re/Lean-QuantumInfo/QuantumInfo/Finite/Pinching.html. Accessed 20 July 2026.

[7] Tianrun Zhao and Nengkun Yu. Formalizing CHSH rigidity in Lean 4, 2026.

[8] Chengkai Zhu, Ziao Tang, Guocheng Zhen, Yimeng Cao, Yusheng Zhao, Ranyiliu Chen, Xuanqiang Zhao, Lei Zhang, and Xin Wang. Lean-QIT: Towards a formal infrastructure for quantum information theory, 2026.